SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 11

Name of Student : K.saikiran
Enrollment No. : 2303A51539
Batch No. : 22

Task Description #1 – Stack Implementation
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.
Sample Input Code:
class Stack: pass
Expected Output:
• A functional stack implementation with all required methods and docstrings.

Prompt:-
Generate a complete Python class for a Stack with push, pop, peek, and is_empty methods. Include proper comments and short docstrings explaining each method's purpose. Ensure the stack follows LIFO behavior and is fully functional.
Code:-

Assignment11.py > ...

```python
#Generate a complete Python class for a Stack with push, pop, peek, and is_empty methods. Include proper comments

class Stack:
    """A simple implementation of a stack data structure following LIFO behavior."""

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an error if the stack is empty."""
        if self.is_empty():
            raise IndexError("Pop from an empty stack.")
        return self.items.pop()

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises an error if the stack is empty."""
        if self.is_empty():
            raise IndexError("Peek from an empty stack.")
        return self.items[-1]

    def is_empty(self):
        """Return True if the stack is empty, False otherwise."""
        return len(self.items) == 0
# Test cases
stack = Stack()
stack.push(1)
stack.push(2)
print(stack.peek())  # Output: 2
print(stack.pop())   # Output: 2
print(stack.is_empty())  # Output: False
print(stack.pop())   # Output: 1
print(stack.is_empty())  # Output: True
try:
    stack.pop()  # This should raise an error
except IndexError as e:
    print(e)  # Output: Pop from an empty stack.
try:
    stack.peek()  # This should raise an error
except IndexError as e:
    print(e)  # Output: Peek from an empty stack.
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON>
2
2
False
1
True
Pop from an empty stack.
Peek from an empty stack.
```

Justification:-
A Stack works on the Last-In-First-Out principle. It is useful in situations like undo operations, expression evaluation, and backtracking. Using a class structure helps organize stack operations clearly and makes the implementation reusable.

Task Description #2 – Queue Implementation
Task: Use AI to implement a Queue using Python lists.
Sample Input Code:
class Queue: pass
Expected Output:
• FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Prompt:-
Write a Python class that implements a Queue using lists. Include enqueue, dequeue, peek, and size methods with clear comments and docstrings. The implementation must follow FIFO order correctly.

Code:-

```python
#Write a Python class that implements a Queue using lists. Include enqueue, dequeue, peek, and size methods with clear

class Queue:
    """A simple implementation of a queue data structure following FIFO behavior."""

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue. Raises an error if the queue is empty."""
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue.")
        return self.items.pop(0)

    def peek(self):
        """Return the item at the front of the queue without removing it. Raises an error if the queue is empty."""
        if self.is_empty():
            raise IndexError("Peek from an empty queue.")
        return self.items[0]

    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)

    def is_empty(self):
        """Return True if the queue is empty, False otherwise."""
        return len(self.items) == 0
# Test cases
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
print(queue.peek())  # Output: 1
print(queue.dequeue())  # Output: 1
print(queue.size())  # Output: 1
print(queue.dequeue())  # Output: 2
print(queue.is_empty())  # Output: True
try:
    queue.dequeue()  # This should raise an error
except IndexError as e:
    print(e)  # Output: Dequeue from an empty queue.
try:
    queue.peek()  # This should raise an error
except IndexError as e:
    print(e)  # Output: Peek from an empty queue.
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON>
1
1
1
2
True
Dequeue from an empty queue.
Peek from an empty queue.
```

Justification:-

A Queue follows First-In-First-Out order, which matches real-world systems like ticket counters and scheduling systems. Implementing it using a list makes it simple and easy to understand for beginners. It clearly demonstrates sequential processing.

SR UNIVERSITY

Task Description #3 – Linked List
Task: Use AI to generate a Singly Linked List with insert and display methods.
Sample Input Code:
class Node: pass
class LinkedList: pass
Expected Output:
• A working linked list implementation with clear method documentation.

Prompt:-
Create a Singly Linked List in Python with Node and LinkedList classes. Implement insert and display methods, and add concise documentation for each method. Ensure the list works dynamically for multiple insertions.
Code:-

```python
#Task-3
# write a python program to implement a singly linked list class with insert, delete, search and display
class Node:
    """A node in a singly linked list."""

    def __init__(self, data):
        """Initialize a node with data and a pointer to the next node."""
        self.data = data
        self.next = None
class SinglyLinkedList:
    """A simple implementation of a singly linked list with basic operations."""

    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def insert(self, data):
        """Insert a new node with the given data at the end of the list."""
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def delete(self, key):
        """Delete the first node with the specified key. Raises an error if the key is not found."""
        current_node = self.head
        previous_node = None

        while current_node and current_node.data != key:
            previous_node = current_node
            current_node = current_node.next
```

```python
#Create a Singly Linked List in Python with Node and LinkedList classes. Implement insert a
class Node:
    """A node in a singly linked list."""

    def __init__(self, data):
        """Initialize a node with the given data and a pointer to the next node."""
        self.data = data
        self.next = None
class LinkedList:
    """A simple implementation of a singly linked list."""

    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def insert(self, data):
        """Insert a new node with the given data at the end of the linked list."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """Display the contents of the linked list."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()  # for a new line after displaying the list
# Test cases
linked_list = LinkedList()
linked_list.insert(1)
linked_list.insert(2)
linked_list.insert(3)
linked_list.display()  # Output: 1 2 3
linked_list.insert(4)
linked_list.display()  # Output: 1 2 3 4
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON>
1 2 3
1 2 3 4
```

Justification:-

A Linked List allows dynamic memory allocation and efficient insertion without shifting elements. It helps understand pointer-based data structures. This structure is useful when frequent insertions and deletions are required.

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable: pass
Expected Output:
• Collision handling using chaining, with well-commented methods.

Prompt:-
Implement a Hash Table in Python with insert, search, and delete operations. Use chaining to handle collisions. Include comments and short docstrings explaining how hashing and collision handling work.
Code:-

```python
#Implement a Hash Table in Python with insert, search, and delete operations. Use chaining to handle collisi
class HashTable:
    """A simple implementation of a hash table using chaining for collision resolution."""

    def __init__(self, size=10):
        """Initialize the hash table with a specified size and create empty buckets."""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate a hash value for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table. If the key already exists, update its value."""
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)  # Update existing key
                return
        bucket.append((key, value))  # Insert new key-value pair

    def search(self, key):
        """Search for a value by its key. Returns the value if found, or None if the key does not exist."""
        index = self._hash(key)
        bucket = self.table[index]
        for k, v in bucket:
            if k == key:
                return v
        return None  # Key not found

    def delete(self, key):
        """Delete a key-value pair from the hash table. Raises an error if the key does not exist."""
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                del bucket[i]  # Remove the key-value pair
                return
        raise KeyError("Key not found.")
# Test cases
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))  # Output: Alice
print(hash_table.search("age"))   # Output: 30
hash_table.insert("name", "Bob")  # Update existing key
print(hash_table.search("name"))  # Output: Bob
hash_table.delete("age")
print(hash_table.search("age"))   # Output: None
try:
    hash_table.delete("age")  # This should raise an error
except KeyError as e:
    print(e)  # Output: Key not found.
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON>
Alice
30
Bob
None
'Key not found.'
```

Justification:-

A Hash Table provides fast data retrieval using keys. Collision handling through chaining ensures multiple values can be stored at the same index. It is widely used in databases, caching systems, and symbol tables.

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display connections.

Prompt:-

write a python program to implement a graph class using an adjacency list with add_vertex, add_edge, remove_vertex, remove_edge and display methods and concise docstring.

Code:-

```
#Develop a Graph class in Python using an adjacency list representation. Include methods to add vertices, add edg
class Graph:
    """A simple implementation of a graph using an adjacency list representation."""

    def __init__(self):
        """Initialize an empty graph with an adjacency list."""
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        """Add a vertex to the graph. If the vertex already exists, do nothing."""
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, vertex1, vertex2):
        """Add an edge between two vertices in the graph. If either vertex does not exist, it will be added."""
        self.add_vertex(vertex1)
        self.add_vertex(vertex2)
        self.adjacency_list[vertex1].append(vertex2)
        self.adjacency_list[vertex2].append(vertex1)  # For undirected graph

    def display(self):
        """Display the adjacency list of the graph."""
        for vertex, edges in self.adjacency_list.items():
            print(f"{vertex}: {', '.join(edges)}")
# Test cases
graph = Graph()
graph.add_edge("A", "B")
graph.add_edge("A", "C")
graph.add_edge("B", "D")
graph.display()
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON>
A: B, C
B: A, D
C: A
D: B
```

Justification:-
Graphs represent networks such as roads, social networks, and communication systems. An adjacency list is memory-efficient for sparse graphs. It clearly shows relationships between connected nodes.

Task Description #6: Smart Hospital Management System – Data Structure Selection
A hospital wants to develop a Smart Hospital Management System that handles:
1. Patient Check-In System – Patients are registered and treated in order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.
Student Task
• For each feature, select the most appropriate data structure from the list below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.
Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:-
Write a python program on Select appropriate data structures for each hospital feature and justify your choices in 2–3 sentences. Then implement one selected feature as a working Python program with comments and docstrings.

Code:-

```python
class PatientCheckIn:
    """A simple implementation of a patient check-in system using a queue."""

    def __init__(self):
        """Initialize an empty queue for patient check-in."""
        self.queue = []

    def check_in(self, patient_name):
        """Add a patient to the check-in queue."""
        self.queue.append(patient_name)
        print(f"{patient_name} has checked in.")

    def attend_patient(self):
        """Attend to the next patient in the queue. Raises an error if the queue is empty."""
        if not self.queue:
            raise IndexError("No patients in the queue")
        patient_name = self.queue.pop(0)
        print(f"Attending to {patient_name}.")

    def display_queue(self):
        """Display the current patients in the check-in queue."""
        if not self.queue:
            print("No patients in the queue.")
        else:
            print("Current patients in the queue:")
            for patient in self.queue:
                print(patient)
# Example usage
check_in_system = PatientCheckIn()
check_in_system.check_in("Alice")
check_in_system.check_in("Bob")
check_in_system.display_queue()
check_in_system.attend_patient()
check_in_system.display_queue()
```

Output:-

```
Alice has checked in.
Bob has checked in.
Current patients in the queue:
Alice
Bob
Attending to Alice.
Current patients in the queue:
Bob
```

Justification:-

Different hospital operations require different data handling methods. For example, queues handle patient flow, priority queues manage emergencies, and hash tables allow quick record retrieval. Choosing correct structures improves efficiency and organization.

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.
Student Task
• For each feature, select the most appropriate data structure from the list below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.
Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:-
write a python program to Choose suitable data structures for each traffic management feature and justify each selection clearly. Implement one selected feature as a complete Python program.

Code:-

```python
class TrafficManagementSystem:
    """A simple implementation of a smart city traffic management system."""

    def __init__(self):
        """Initialize the traffic management system with queues, priority queues, graphs, and hash tables."""
        self.traffic_queue = []
        self.emergency_queue = []
        self.city_graph = {}
        self.vehicle_registration = {}

    def add_vehicle_to_traffic(self, vehicle):
        """Add a vehicle to the traffic queue."""
        self.traffic_queue.append(vehicle)
        print(f"{vehicle} has been added to the traffic queue.")

    def add_emergency_vehicle(self, vehicle):
        """Add an emergency vehicle to the priority queue."""
        self.emergency_queue.append(vehicle)
        print(f"{vehicle} has been added to the emergency queue.")

    def add_road(self, from_location, to_location):
        """Add a road between two locations in the city graph."""
        if from_location not in self.city_graph:
            self.city_graph[from_location] = []
        if to_location not in self.city_graph:
            self.city_graph[to_location] = []
        self.city_graph[from_location].append(to_location)
        self.city_graph[to_location].append(from_location)  # For undirected graph
        print(f"Road added between {from_location} and {to_location}.")

    def register_vehicle(self, license_plate, owner_name):
        """Register a vehicle in the hash table."""
        self.vehicle_registration[license_plate] = owner_name
        print(f"Vehicle with license plate {license_plate} registered under {owner_name}.")
# Example usage
traffic_system = TrafficManagementSystem()
traffic_system.add_vehicle_to_traffic("Car A")
traffic_system.add_vehicle_to_traffic("Car B")
traffic_system.add_emergency_vehicle("Ambulance")
traffic_system.add_road("Intersection 1", "Intersection 2")
traffic_system.add_road("Intersection 2", "Intersection 3")
traffic_system.register_vehicle("ABC123", "John Doe")
```

Output:-

```
Car A has been added to the traffic queue.
Car B has been added to the traffic queue.
Ambulance has been added to the emergency queue.
Road added between Intersection 1 and Intersection 2.
Road added between Intersection 2 and Intersection 3.
Vehicle with license plate ABC123 registered under John Doe.
PS C:\AIAC LAB>
```

Justification:-

Traffic systems require ordered processing, priority handling, and fast lookups. Using structures like queues, priority queues, graphs, and hash tables ensures efficient real-time management. Correct selection improves performance and scalability.

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:
1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.
Student Task
• For each feature, select the most appropriate data structure from the list below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.
Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:-
write a python program to Select the most appropriate data structures for each e-commerce feature and justify your decisions briefly. Implement one selected feature with proper documentation.

Code:-

```python
class ECommercePlatform:
    """A simple implementation of a smart e-commerce platform."""

    def __init__(self):
        """Initialize the e-commerce platform with linked lists, queues, hash tables, and graphs."""
        self.product_list = []
        self.order_queue = []
        self.customer_reviews = {}
        self.product_graph = {}

    def add_product(self, product_name):
        """Add a product to the linked list."""
        self.product_list.append(product_name)
        print(f"Product '{product_name}' added to the catalog.")

    def remove_product(self, product_name):
        """Remove a product from the linked list. Raises an error if the product is not found."""
        if product_name in self.product_list:
            self.product_list.remove(product_name)
            print(f"Product '{product_name}' removed from the catalog.")
        else:
            raise ValueError("Product not found in the catalog")

    def place_order(self, order):
        """Place an order and add it to the queue."""
        self.order_queue.append(order)
        print(f"Order '{order}' placed.")

    def add_review(self, product_id, review):
        """Add a customer review for a product in the hash table."""
        if product_id not in self.customer_reviews:
            self.customer_reviews[product_id] = []
        self.customer_reviews[product_id].append(review)
        print(f"Review added for product ID '{product_id}'.")

    def add_product_relationship(self, product1, product2):
        """Add a relationship between two products in the graph."""
        if product1 not in self.product_graph:
            self.product_graph[product1] = []
        if product2 not in self.product_graph:
            self.product_graph[product2] = []
        self.product_graph[product1].append(product2)
        self.product_graph[product2].append(product1)  # For undirected graph
        print(f"Relationship added between '{product1}' and '{product2}'.")
# Example usage
ecommerce_platform = ECommercePlatform()
ecommerce_platform.add_product("Laptop")
ecommerce_platform.add_product("Smartphone")
ecommerce_platform.place_order("Order 1")
ecommerce_platform.add_review("Laptop", "Great product!")
ecommerce_platform.add_product_relationship("Laptop", "Smartphone")
```

Output:-

```
Product 'Laptop' added to the catalog.
Product 'Smartphone' added to the catalog.
Order 'Order 1' placed.
Review added for product ID 'Laptop'.
Relationship added between 'Laptop' and 'Smartphone'.
PS C:\AIAC LAB>
```

Justification:-

E-commerce platforms handle dynamic cart updates, order processing, ranking systems, and quick product searches. Different data structures support different needs like dynamic storage, priority ranking, and fast retrieval. Proper structure selection ensures smooth system performance.